



INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Linguagem C/C++

Funções

Funções

- Um aspecto importante na resolução de um problema complexo é conseguir dividi-lo em subproblemas menores.
- Sendo assim, ao criarmos um programa para resolver um determinado problema, uma tarefa importante é dividir o código em partes menores, fáceis de serem compreendidas e mantidas.
- Funções são estruturas da linguagem que agrupam um conjunto de comandos, que são executados quando a função é chamada.

Exemplo de uma chamada de função: `scanf("%d", &x);`

- Funções podem retornar um valor ao final de sua execução.

Exemplo de uma função que retorna um valor: `x = sqrt(4);`

Porque usar funções?

- Evitar que os blocos do programa fiquem grandes demais e, por consequência, difíceis de ler e entender.
- Separar o programa em partes que possam ser logicamente compreendidas de forma isolada.
- Permitir o reaproveitamento de códigos, implementados por você ou por outros programadores.
- Evitar que um trecho de código seja repetido várias vezes dentro de um mesmo programa, evitando inconsistências e facilitando alterações.

Definindo uma função:

```
tipo nome(tipo parametro_1, ..., tipo parâmetro_N) {  
    comandos;  
    ...  
    comandos;  
  
    return valor_de_retorno;  
}
```

- Toda função deve ter um tipo que determina seu valor de retorno.
- Os parâmetros são variáveis que serão utilizadas pela função. Tais variáveis são inicializadas com valores na chamada da função.
- Para os nomes das funções e dos seus parâmetros valem as mesmas regras para os nomes de variáveis.
- Não é possível definir uma ou mais funções no mesmo programa com o mesmo nome.

Exemplos:

A função abaixo soma dois valores, passados como parâmetros:

```
int soma(int a, int b) {  
    int c;  
    c = a + b;  
    return c;  
}  
  
int main() {  
    int res, x1 = 4, x2 = -10;  
    res = soma(5, 6);  
    printf("Primeira soma: %d\n", res);  
    printf("Segunda soma: %d\n", soma(x1, x2));  
    return 0;  
}
```

- O valor de retorno deve ser do mesmo tipo definido para a função.
- Quando o comando return é executado, a função termina sua execução e retorna o valor indicado para quem fez a chamada da função.
- Importante: a execução de um programa sempre começa pela função main.

Função sem parâmetro:

- Uma função pode não ter parâmetros, neste caso, basta não informá-los:
- Em C, funções só podem ser definidas fora de outras funções.
- A expressão contida no comando return é chamada de valor de retorno. Após a execução do comando return, nenhum outro comando da função será executado.

```
#include <stdio.h>

int leNumero() {
    int n;
    printf("Digite um numero: ");
    scanf("%d", &n);
    return n;
}

int soma(int a, int b) {
    return (a + b);
    printf("bla bla bla bla!"); /* Nao imprime esta mensagem */
}

int main() {
    int x1, x2;
    x1 = leNumero();
    x2 = leNumero();
    printf("Valor da soma: %d\n", soma(x1, x2));

    return 0;
}
```

Chamada de função:

- Para cada um dos parâmetros da função, devemos fornecer um valor, de mesmo tipo, na chamada da função.
- Ao chamar uma função passando variáveis como parâmetros, estamos usando apenas os seus valores que serão copiados para as variáveis parâmetros da função.
- Os valores das variáveis usadas na chamada da função não são afetados por alterações dentro da função.

```
#include <stdio.h>
```

```
int somaEsquisita(int x, int y) {  
    x = x + 1;  
    y = y + 1;  
    return (x + y);  
}
```

```
int main() {  
    int a = 10, b = 5;  
  
    printf("Soma de a e b: %d\n", a + b);  
    printf("Soma de x e y: %d\n", somaEsquisita(a, b));  
    printf("a: %d\n", a);  
    printf("b: %d\n", b);  
  
    return 0;  
}
```

Função main()

- O programa principal é uma função especial (main), que possui um tipo fixo (int) e é invocada automaticamente pelo sistema operacional quando este inicia a execução do programa.
- O comando return da função main informa ao sistema operacional se o programa funcionou corretamente ou não. O padrão é que um programa retorne zero, caso tenha funcionado corretamente, ou qualquer outro valor, caso contrário.

Exemplo:

```
#include <stdio.h> int  
main() {  
    printf("Hello, World!\n");  
    return 0;  
}
```


O tipo void

- O tipo void é um tipo especial.
- Este tipo é utilizado para indicar que uma função não retorna nenhum valor.
- Por exemplo, a função abaixo verifica se um número inteiro fornecido como parâmetro é par ou ímpar, e imprime uma mensagem correspondente.
- Podemos usar o comando return (sem qualquer valor) para indicar explicitamente o fim de uma função do tipo void.

```
#include <stdio.h>

void parOuImpar(int numero) {
    if (numero % 2)
        printf("Impar\n");
    else
        printf("Par\n");
    return;//opcional
}

int main() {
    parOuImpar(10);
    parOuImpar(21);

    return 0;
}
```

Definindo funções depois do main

- Até o momento, aprendemos definir as funções antes do programa principal (função main).
- O que ocorreria se definíssemos uma função depois da função main?
- Ocorreria um erro de compilação.

```
#include <stdio.h>
```

```
int main() {  
    float a = 8, b = 5;  
    printf("%f\n", soma(a, b)); /* Erro funcao  
    desconhecida */  
    return 0;  
}
```

```
float soma(  
float op1, float op2){  
    return (op1 +  
        op2);  
}
```

Declarar a função sem defini-la:

- Para organizar melhor um programa e podermos implementar funções em partes distintas do arquivo, *protótipos de funções* são utilizados.
- Protótipos de funções correspondem à primeira linha da definição de uma função contendo tipo de retorno, nome da função, parâmetros e um ponto e vírgula.

tipo nome(tipo parametro1, ..., tipo parametroN);

- O protótipo de uma função deve vir sempre antes do seu uso.
- É comum colocar os protótipos de funções no início do arquivo do programa.

```
#include <stdio.h>
```

```
float soma(float op1, float op2);  
float subtr(float op1, float op2);
```

```
int main() {  
    float a = 8, b = 5;  
    printf("%f\n %f\n", soma(a, b), subtr(a, b));  
    return 0;  
}
```

```
float soma(float op1, float op2) {  
    return (op1 + op2);  
}
```

```
float subtr(float op1, float op2) {  
    return (op1 - op2);  
}
```

Exemplo num primos:

- Criar uma função que calcule os n primeiros números primos;
 - Se o número de primos a serem calculados for negativo, usaremos o valor absoluto deste.
 - Podemos criar uma função que testa se um número é primo ou não (note que este é um bloco logicamente bem definido).
 - Vamos criar também uma função que retorna o valor absoluto de um número.
- Depois fazemos chamadas para estas funções.

```

#include <stdio.h>
int testaPrimo(int candidato);
int valorAbs(int x);

int main() {
    int n, candidato = 2, primos = 0;

    printf("Numero de primos a serem calculados: ");
    scanf("%d", &n);
    n = valorAbs(n);

    while (primos < n) {
        if (testaPrimo(candidato)) {
            printf("%d\n", candidato);
            primos++;
        }
        candidato++;
    }
    return 0;
}

```

```

/* Calcula o valor absoluto de x */ int valorAbs(int x) {
    if (x < 0)
        return -x; else
        return x;
}

```

```

/* Verifica se um número candidato eh primo */
int testaPrimo(int candidato) {
    int primo = 1, divisor;

    for (divisor = 2; primo && (divisor <= candidato/2); divisor++)
        if ((candidato % divisor) == 0)
            primo = 0;

    if (primo)
        return 1; /* Se for primo, retorna 1 (verdadeiro) */
    else
        return 0; /* Se nao for, retorna 0 (falso) */
}

```

Variável Global e Local

- Uma variável é chamada local se ela foi declarada dentro de uma função. Nesse caso, ela existe somente dentro daquela função e, após o término da execução da mesma, a variável deixa de existir.
- Variáveis utilizadas como parâmetros de funções também são locais.
- Uma variável é chamada global se ela for declarada fora de qualquer função. Essa variável é visível em todas as funções. Qualquer função pode alterá-la e ela existe durante toda a execução do programa.
- Variáveis globais dificultam a legibilidade, manutenção e reuso de funções e, por estes motivos, devem ser evitadas.

#Declaração de bibliotecas
#Definição de constantes Protótipos de funções;

Declaração de variáveis globais;
int main() {
 Declaração de variáveis locais;
 Comandos;
}

int funcao1(parametros) {
 Declaração de variáveis locais;
 Comandos;
}

int funcao2(parametros) {
 Declaração de variáveis locais;
 Comandos;
}

Escopo de variável:

- O escopo de uma variável determina em quais partes do código pode-se ter acesso a ela.
- A regra de escopo em C é bem simples:
 - ↳ As variáveis globais são visíveis por todas as funções.
 - ↳ As variáveis locais são visíveis apenas na função onde foram declaradas.
- É possível declarar variáveis locais com o mesmo nome de variáveis globais. Nesta situação, a variável local “esconde/sobrepõe” a variável global.

```
#include <stdio.h>
```

```
void funcao1();
```

```
int funcao2(int local_b);
```

```
int global_a;
```

```
int main() {
```

```
    int local_main;
```

```
    /* Neste ponto sao visiveis global_a e local_main */
```

```
}
```

```
void funcao1() {
```

```
    int local_a;
```

```
    /* Neste ponto sao visiveis global_a e local_a */
```

```
}
```

```
int funcao2(int local_b) {
```

```
    int local_c;
```

```
    /* Neste ponto sao visiveis global_a, local_b e local_c */
```

```
}
```

Vetores e funções:

- Vetores também podem ser passados como parâmetros em funções.
- Ao contrário dos tipos simples, vetores têm um comportamento diferente quando usados como parâmetros de funções.
- Quando uma variável simples é passada como parâmetro, seu valor é atribuído para uma nova variável local da função.
- No caso de vetores, não é criado um novo vetor.
- Isto significa que os valores de um vetor podem ser alterados dentro de uma função.
- Para indicar que um parâmetro é um vetor, usamos [] na frente do nome do parâmetro.

```
int max(int vetor[], int tam) {
```

```
...
```

```
}
```

- Ao chamar uma função que possui um vetor como parâmetro, devemos apenas indicar o nome do vetor a ser fornecido para a função (sem usar [] na frente do nome do vetor).

Exemplo:

```
int elementos[10], n;
```

```
...
```

```
x = max(elementos, n);
```



```
#include <stdio.h>
```

```
void funcao(int vet[], int tam) {  
    int i;  
    for (i = 0; i < tam; i++)  
        vet[i] = 5;  
}
```

```
int main() {  
    int x[10], i;  
  
    for (i = 0; i < 10; i++)  
        x[i] = 8;  
  
    funcao(x, 6);  
  
    for (i = 0; i < 10; i++)  
        printf("%d ", x[i]);  
  
    return 0;  
}
```

Vetores não podem ser usados como retorno de uma função, mas podem ser alterados dentro dela:

```
#include <stdio.h>
```

```
void leVetor(int vet[], int tam) {
```

```
    int i;
```

```
    for (i = 0; i < tam; i++) {
```

```
        printf("Digite numero: ");
```

```
        scanf("%d", &vet[i]);
```

```
    }
```

```
}
```

```
void imprimeVetor(int vet[], int tam) {
```

```
    int i;
```

```
    for (i = 0; i < tam; i++)
```

```
        printf("vet[%d] = %d\n", i, vet[i]);
```

```
}
```

```
int main() {
```

```
    int vet1[10], vet2[20];
```

```
    printf("----- Vetor 1 -----\\n");
```

```
    leVetor(vet1, 10);
```

```
    printf("----- Vetor 2 -----\\n");
```

```
    leVetor(vet2, 20);
```

```
    printf("----- Vetor 1 -----\\n");
```

```
    imprimeVetor(vet1, 10);
```

```
    printf("----- Vetor 2 -----\\n");
```

```
    imprimeVetor(vet2, 20);
```

```
    return 0;
```

```
}
```

Matrizes e funções

- Ao passar um vetor como parâmetro, não é necessário fornecer o seu tamanho na declaração da função.
- Quando usamos uma matriz, a possibilidade de não informar o tamanho na declaração se restringe à primeira dimensão.
- Como vimos anteriormente, matrizes são de fato alocadas pelo compilador como vetores, por isso é preciso informar todas as dimensões (com exceção, eventualmente, da primeira) para que o compilador seja capaz de determinar corretamente os elementos de uma matriz passada como parâmetro para uma função.
- Assim como vetores, matrizes passadas como parâmetros podem ser modificadas dentro das funções.

função com matriz

- Pode-se criar uma função deixando de indicar a primeira dimensão:

```
void mostra_matriz(int mat[][10], int linhas) {  
    ...  
}
```

- Ou pode-se criar uma função indicando todas as dimensões:

```
void mostra_matriz(int mat[5][10], int linhas) {  
    ...  
}
```

- Não se pode deixar de indicar outras dimensões (exceto a primeira):

```
void mostra_matriz(int mat[5][], int linhas) {  
    /* Esta funcao nao compila */  
    ...  
}
```

```
#include <stdio.h>

void imprime_matriz(int linhas, int colunas, int matriz[][10]) {
    int i, j;
    for (i = 0; i < linhas; i++) {
        for (j = 0; j < colunas; j++)
            printf("%2d ", matriz[i][j]);
        printf("\n");
    }
}

int main() {
    int matriz[8][10] = { { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9},
                           {10, 11, 12, 13, 14, 15, 16, 17, 18, 19},
                           {20, 21, 22, 23, 24, 25, 26, 27, 28, 29},
                           {30, 31, 32, 33, 34, 35, 36, 37, 38, 39},
                           {40, 41, 42, 43, 44, 45, 46, 47, 48, 49},
                           {50, 51, 52, 53, 54, 55, 56, 57, 58, 59},
                           {60, 61, 62, 63, 64, 65, 66, 67, 68, 69},
                           {70, 71, 72, 73, 74, 75, 76, 77, 78, 79} };

    imprime_matriz(8, 10, matriz);
    return 0;}
```

Recursão

- Uma função pode chamar a si própria. A isso chamamos recursividade, que é o processo de repetir alguma coisa de maneira similar. Uma função assim é chamada de função recursiva.
- A recursão também é chamada de definição circular. Ela ocorre quando algo é definido em termos de si mesmo.
- Como calcular o fatorial de 4 (definido como 4!)?

$$0! = 1$$

$$N! = N * (N - 1)!$$

```
int fatorial (int n){  
    if(n == 0)  
        return 1;  
    else  
        return n*fatorial(n-1);  
}
```